

SimpleFRP

Project Guide and Linux Installation Manual

Version v0.1.0 - GitHub: <https://github.com/QiaoxiuLi/SimpleFRP>

Purpose	Fast TCP reverse forwarding with minimal setup
Default ports	Dashboard 8387, control 8388, public tunnels 20000-60000
Primary platforms	Linux server/client, macOS client, Windows client
Release page	https://github.com/QiaoxiuLi/SimpleFRP/releases/latest

1. Project Overview

SimpleFRP is a lightweight TCP reverse forwarding tool. It is intended for users who need quick deployment and do not want to write FRP configuration files manually. A public server accepts authenticated client control connections and exposes public TCP ports that forward back to services running on the client machine.

- One primary binary: simplefrp.
- Server and client roles are selected by role files or daemon flags.
- First tunnel is created automatically during client setup.
- Dashboard is embedded and requires no external web runtime.
- Packages configure autostart and clean uninstall behavior.

2. Linux Installation From GitHub

Ubuntu 22.04 or newer

```
wget https://github.com/QiaoxiuLi/SimpleFRP/releases/download/v0.1.0/simplefrp-server-linux-amd64.deb
sudo dpkg -i simplefrp-server-linux-amd64.deb
sudo simplefrp pwd

wget https://github.com/QiaoxiuLi/SimpleFRP/releases/download/v0.1.0/simplefrp-client-linux-amd64.deb
sudo dpkg -i simplefrp-client-linux-amd64.deb
simplefrp set server
```

Rocky Linux 9, RHEL 9, AL2023, AliLinux, HCE

```
curl -LO https://github.com/QiaoxiuLi/SimpleFRP/releases/download/v0.1.0/simplefrp-server-linux-amd64.rpm
sudo rpm -i simplefrp-server-linux-amd64.rpm
sudo simplefrp pwd

curl -LO https://github.com/QiaoxiuLi/SimpleFRP/releases/download/v0.1.0/simplefrp-client-linux-amd64.rpm
sudo rpm -i simplefrp-client-linux-amd64.rpm
simplefrp set server
```

The public server firewall or cloud security group must allow TCP ports 8388, 8387, and any public tunnel ports that clients will use.

3. Daily Operation

Command	Use
sudo simplefrp pwd	Set or reset the server password and start or restart the server daemon.
simplefrp set server	Configure client server address/password and create the first tunnel.
simplefrp state	Show tunnel ID, local port, public port, and success/failed status.
simplefrp next	Create another tunnel.
simplefrp 0 local 25001	Change tunnel 0 local target port.
simplefrp 0 public 35001	Change tunnel 0 public server port.
simplefrp delete 0	Delete tunnel 0.
simplefrp uninstall	Stop services, remove autostart, clean files, and release processes.

4. Functional Implementation

Server daemon

The server daemon loads `server.toml`, opens the control listener on 8388, opens the dashboard on 8387, manages connected client sessions, listens on public tunnel ports, and bridges external TCP connections to client data connections.

Client daemon

The client daemon loads `client.toml`, maintains a control connection to the server, registers configured tunnels, opens a data connection when requested, and connects that stream to 127.0.0.1.

Tunnel flow

- Client creates or registers a tunnel with local and public port information.
- Server reserves and listens on the public port.
- External traffic reaches the server public port.
- Server sends an `open_stream` request to the client over the control connection.
- Client opens a data connection and connects to its local TCP service.
- Server and client copy bytes in both directions until the connection closes.

Dashboard and storage

The dashboard uses embedded static files and Go `net/http`. `bbolt` stores traffic counters, connection logs, and tunnel records. APIs are read-only and designed for lightweight inspection.

5. Security Model

- Plaintext passwords are never stored.
- Server passwords use Argon2id hashing.
- Client password material is derived and stored without the original password.
- Control requests use HMAC-SHA256 signatures; the derived client secret is not sent over the network.
- Dashboard APIs are read-only.

6. Autostart and Uninstall

Linux packages create a `simplefrp` system user, install a `systemd` unit, enable autostart, and wait for first configuration before clean runtime start. macOS uses `launchd` where applicable and Windows uses scheduled login startup where applicable.

The uninstall command stops services, disables autostart, removes service files or platform autostart entries, kills remaining SimpleFRP processes except the current uninstall process, and deletes runtime configuration, data, and logs.

7. Third-Party Libraries and Dependencies

Dependency	Purpose
github.com/fatedier/frp v0.62.1	Upstream FRP dependency and integration boundary.
github.com/spf13/cobra v1.8.1	Command line framework.
github.com/spf13/viper v1.19.0	TOML configuration loading and decoding.
go.etcd.io/bbolt v1.3.11	Embedded key/value store for runtime data.
golang.org/x/crypto v0.37.0	Argon2id and crypto helpers.
GoReleaser v2	Cross-platform release build automation.
nFPM	Debian and RPM package generation through GoReleaser.

8. Build, Test, and Recovery

```
git clone https://github.com/QiaoxiuLi/SimpleFRP.git
cd SimpleFRP
make test
make release
```

Runtime files are not committed to git. Back up production Linux runtime state separately:

```
sudo tar -czf simplefrp-runtime-backup.tar.gz /etc/simplefrp /var/lib/simplefrp
sudo tar -xzf simplefrp-runtime-backup.tar.gz -C /
sudo systemctl restart simplefrp-server || sudo systemctl restart simplefrp-client
```

9. Troubleshooting

Error	Action
SERVER_UNREACHABLE	Check server address, service status, firewall/security group, and TCP port 8388.
AUTH_FAILED	Run simplefrp set server again and enter the correct password.
LOCAL_PORT_IN_USE	Choose another local port.
SERVER_PORT_IN_USE	Choose another public port or free it on the server.
INVALID_PASSWORD_FORMAT	Use at least 8 letters or numbers. Do not use spaces, symbols, or Chinese characters.